PROJECT REPORT · KNOWLEDGE TRACING FOR MATHS PRACTICE

GatedKT: an adaptive maths practice system with a gated knowledge-tracing model

This report explains what was built, how the parts connect, and how the model turns a student's past answers into a prediction for the next question. It ends with the evidence that the work is real and how to check it yourself.

30 October 2026 Repository: aiproj Dataset: ASSISTments 2009 skill-builder (public)

SUMMARY

GatedKT is a complete, working adaptive-practice system: a trained neural knowledge-tracing model served behind a web app with a student quiz, hints and second attempts, a mastery profile and a teacher dashboard. On 313 held-out students the full gated model reaches a test AUC of **0.8713**, the highest of the six models compared. It is statistically better than DKT (GRU), SAKT-lite, BKT and PFA. Across 50 retraining seeds the learned gate improved accuracy on **50 of 50** matched runs. The system is covered by **289 passing automated tests**, and every reported number is traced to a generated result file.

1 What the system does

A student answers maths questions in the browser. Before each question, a neural network estimates the probability that the student will answer each candidate topic correctly. The system uses those estimates to pick the next topic: the weakest one the student is allowed to see. A difficulty policy picks the level (Easy, Medium or Hard). After the session, each topic gets a mastery status with the evidence behind it. A teacher dashboard shows the whole class as a students × topics grid.

The model at the centre, **GatedKT**, keeps two memories of the student: a short-term one built from the last 10 answers and a long-term one built from the whole history. A learned gate decides, for every one of its 64 dimensions, how much to trust each memory. The gate's decision also uses a graph of related skills and the student's help-seeking signals: time taken, number of attempts and hints.

3,119	338,210	123	76,497	0.8713
students (2,495 train · 311 validation · 313 test)	answers in the model's sequences	skills; 1,880 directed edges in the skill graph	trainable parameters in the served model	test AUC, mean of 3 trained checkpoints

2 System architecture

The project has two halves. The **offline pipeline** runs once: it cleans the dataset, builds the skill graph, trains the model and writes the benchmark reports. The **online app** runs as three processes: a browser front end, a Node.js gateway and a Python service that holds the model.

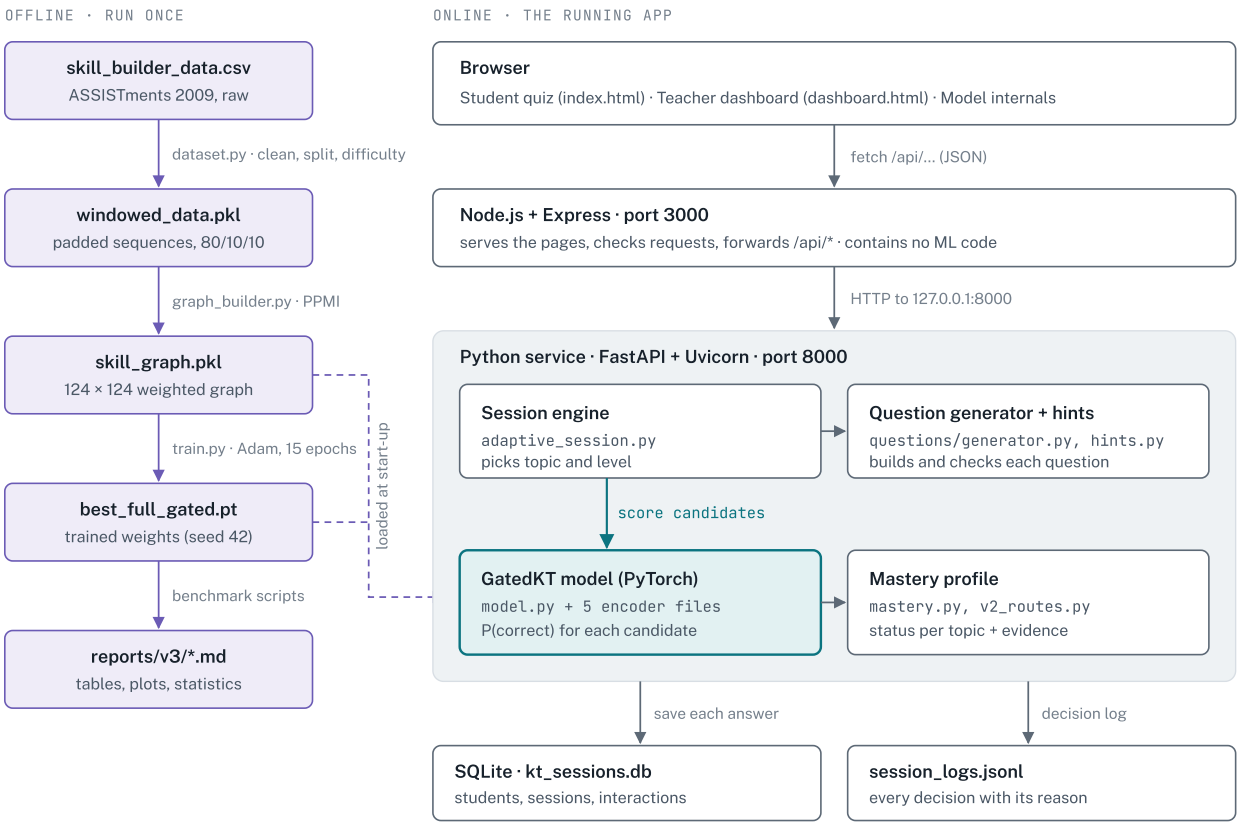


Figure 1. Purple boxes are the offline pipeline. The dashed purple line shows the service loading the skill graph and the trained weights when it starts. The highlighted box is the model. The browser never talks to Python directly: every call goes through the Node gateway, which only checks and forwards requests. The service writes each answer to SQLite and each decision, with its reason, to a log file.

3 Technology stack

All versions are the ones pinned in `requirements.txt` and `server/package.json`. No paid service or external API is used; everything runs on one laptop.

Layer	Technology	What it does here
MODEL	PyTorch 2.14.0	GRUs, embeddings, the gate, training loop and inference
MODEL	PyTorch Geometric 2.8.0	`GATConv` graph-attention layer over the skill graph
DATA	pandas 3.0.5 · NumPy 2.4.6	Cleaning the raw CSV, splitting students, padding sequences, building the graph
EVALUATION	scikit-learn 1.9.1 · SciPy 1.17.1	AUC, paired t-tests and Wilcoxon tests, bootstrap intervals
EVALUATION	Matplotlib 3.11.2	Box plots, histograms and the gate plot in the reports
SERVICE	FastAPI 0.143.0 · Uvicorn 0.54.0 · Pydantic 2.14	Python HTTP API on port 8000; validates every request and response
STORAGE	SQLite (Python `sqlite3`) · JSONL log	Students, sessions and every answer; a decision log per question
GATEWAY	Node.js · Express 4 · cors · dotenv	Serves the web pages on port 3000 and forwards `/api/*` to Python
FRONT END	HTML, CSS, plain JavaScript · Chart.js · vis-network	Quiz and teacher dashboard; Chart.js draws the prediction and level charts, vis-network draws the skill graph
QUALITY	pytest 9.1.1 · Node test scripts · Git	Automated tests, version history of every change

4 Module-wise connectivity

Figure 2 shows which source file calls which. I drew it from the `import` statements in the code. Each box is a real file you can open.

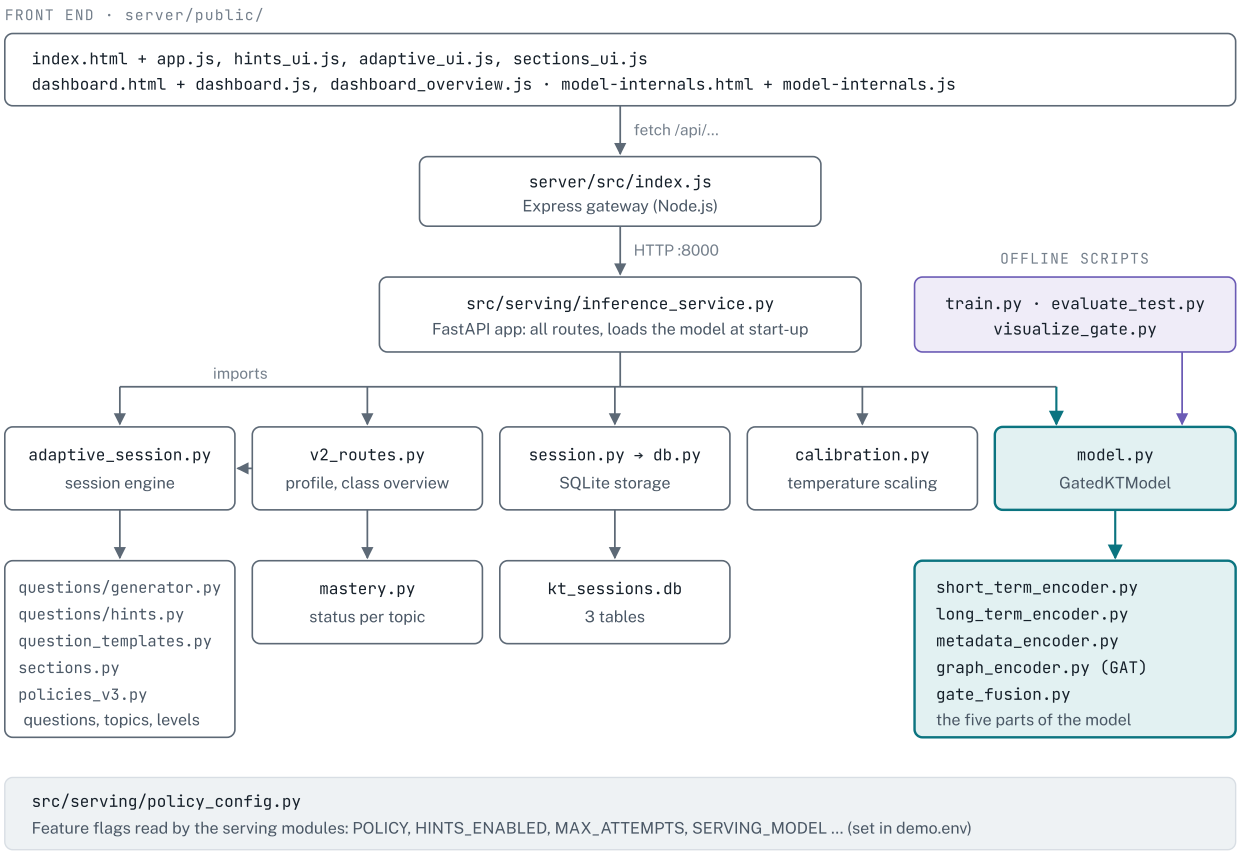


Figure 2. Arrows mean "imports and calls". The service is the only process that loads the model. The training scripts (purple) import the same `model.py`, so the model that is trained is the same code that is served. `v2_routes.py` also reuses the session engine. Every new feature is behind a flag in `policy_config.py`; with no flags set, the system behaves as the original version.

Module	Input → output	Connects to
`src/dataset.py`	raw CSV → `windowed_data.pkl` (padded sequences, difficulty buckets, 80/10/10 split by student, seed 42)	graph_builder.py, train.py
`src/graph_builder.py`	training sequences → `skill_graph.pkl` (PPMI weights; skill pairs seen together in fewer than 5 windows are dropped)	graph_encoder.py

Module	Input → output	Connects to
`src/model.py`	history tensors → P(correct) and gate values; `predict_candidates()` scores possible next questions	the five encoder files
`src/train.py`	data + graph → best-validation checkpoint (`best_<condition>_seed<s>.pt`)	model.py
`src/serving/inference_service.py`	HTTP requests → predictions, questions, profiles; loads the checkpoint and graph once	model.py, adaptive_session.py, session.py, calibration.py, v2_routes.py
`src/serving/adaptive_session.py`	answers so far → next topic and level, generated question, hint, retry	questions/, sections.py, policies_v3.py, policy_config.py
`src/serving/mastery.py`	predictions at Easy/Medium/Hard + answers → status per topic with evidence counts	v2_routes.py
`src/serving/db.py`	answers → SQLite rows (students, sessions, interactions)	session.py
`src/baselines/`	PFA, BKT, DKT, SAKT and others behind one interface, for the benchmarks	benchmark scripts in scripts/
`server/src/index.js`	browser calls → forwarded calls; returns 502/503 when Python is down	inference_service.py

5 How the model works

For each student, the model reads the answers in order and, at every step, predicts whether the *next* answer will be correct. Figure 3 follows one prediction from the inputs to the output. All sizes are those of the served checkpoint.

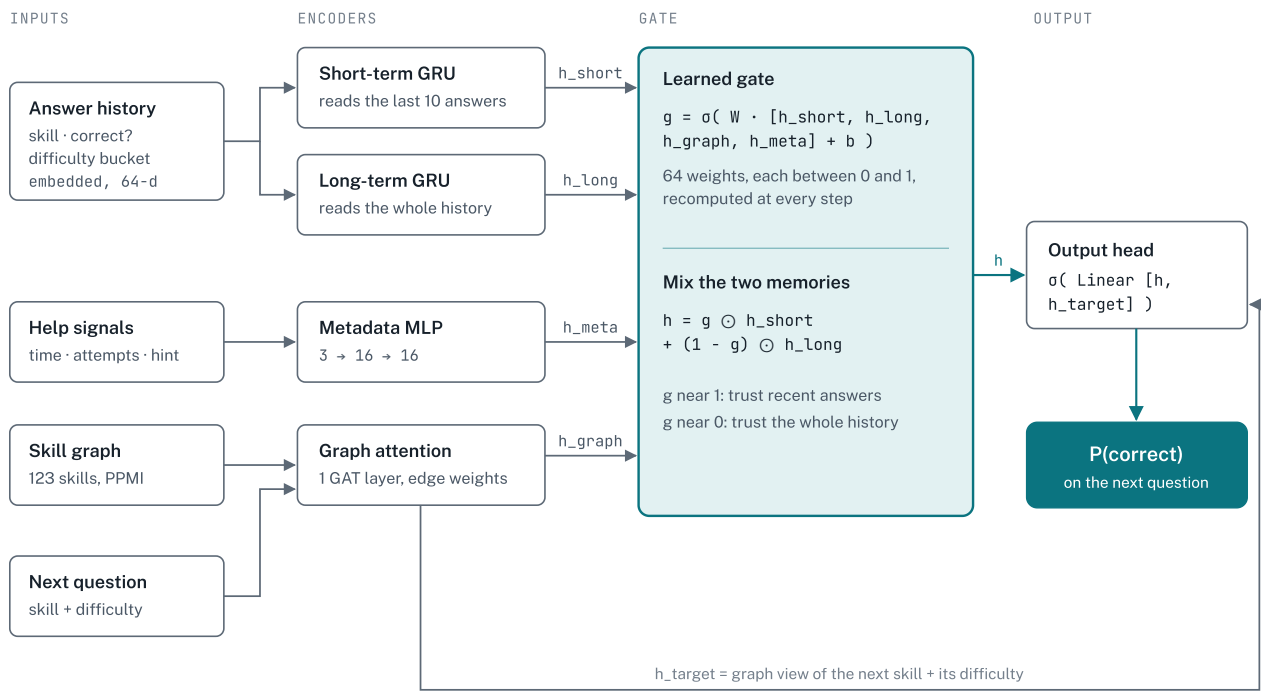


Figure 3. The only place the two memories are combined is the gate (highlighted). The skill graph does two jobs: it gives the gate context about the next skill (h_graph), and it tells the output head *which* skill is being predicted (h_target). If the gate is removed, the code replaces it with a plain average of the three states. That version is the "graph, no gate" model in the experiments.

The computation at step t, as written in the code

```
# src/model.py and the five encoder files
x_t      = E_skill[skill_t] + E_correct[correct_t] + E_diff[bucket_t]     # 64-d
h_short  = GRU_short( x_{t-9} … x_t )          # fresh run over a 10-step window
h_long   = GRU_long ( x_1 … x_t )              # runs over the full history
h_meta   = MLP( log-time, attempts - 1, hint )  # 3 → 16 → 16, fixed normalisation
h_graph  = GAT( E_skill, skill graph )[skill_{t+1}]
g        = sigmoid( W_g · [h_short, h_long, h_graph, h_meta] + b_g )      # 64 values
h        = g ⊙ h_short + (1 - g) ⊙ h_long
p_{t+1}  = sigmoid( w · [h, h_graph + E_diff[bucket_{t+1}]] + b )
```

Where the 76,497 parameters sit: the two GRUs 24,960 each, the gate and output head 13,505, the skill embedding 7,936, the graph layer 4,416, the metadata MLP 336, and the correct and difficulty embeddings 384.

Training

- **Loss:** binary cross-entropy on every real answer; padded positions are masked out.
- **Optimiser:** Adam, learning rate 1e-3, batch 32, 15 epochs. The weights with the best validation AUC are kept.

- **No test leakage:** difficulty buckets and the skill graph come from the 2,495 training students only. The 313 test students are used only for the final scores.
- **Seeds:** three seeds (42, 123, 2024) for each of the three variants. The served model is seed 42.

Serving extras

- **Calibration:** temperature scaling makes the probabilities more honest without changing their order (expected calibration error 0.0249 → 0.0117 on validation data).
- **Hints and retries:** a hint request and the number of attempts are stored with each answer and fed back into the metadata encoder. An answer counts as correct only if it was right on the first try with no hint, the same rule as the dataset.

6 One question, end to end

Figure 4 shows what happens between the student pressing "Next" and the answer being saved.

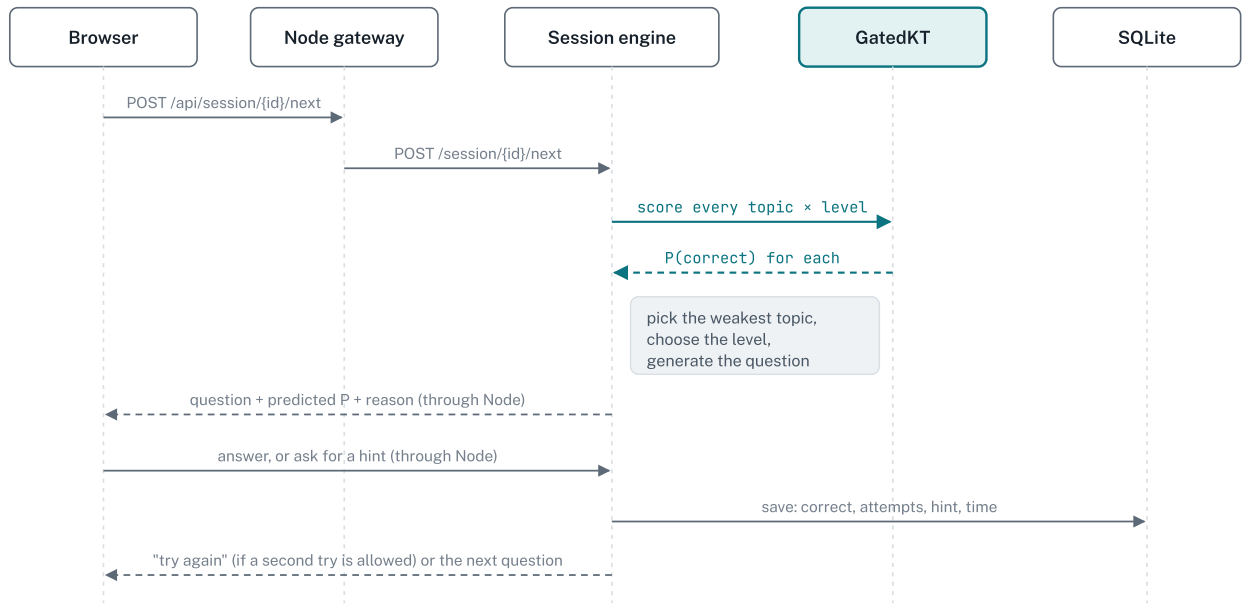


Figure 4. The prediction is made **before** the student sees the question, and it is stored with the question, so it can later be compared with the real answer. Dashed arrows are replies. The model is only asked for scores; it never decides on its own. The session engine applies the rules: no more than 2 questions in a row on one topic, a cap per topic, and the configured difficulty policy.

A full session

1. The student logs in and picks a learner level. Answers from earlier sessions are loaded, so a returning student gets no blind warm-up.
2. Main questions follow the loop in Figure 4. In the demo setting, the topic score blends the model's prediction with the student's own record on that topic.
3. One or two probe questions test topics never asked. For these, the prediction comes from the skill graph and is labelled "predicted, not measured".

4. The profile page shows a status per topic (for example Proficient or Mastered), only where enough answers back it up.

5. The teacher dashboard combines every student into a status grid and a "needs attention" list.

7 Results

The table below shows test AUC on the 313 held-out students: the chance that the model ranks a correct answer above a wrong one, where 0.5 is guessing. The numbers are copied from `reports/v3/benchmark.md`.

Model	Type	Seeds	Test AUC	Full gated vs this model
GatedKT, full gated	ours	3	0.8713	—
GatedKT, graph without gate	ours (ablation)	3	0.8657	better, +0.0056
DKT (GRU)	deep baseline	10	0.8583	better, +0.0130
SAKT-lite	deep baseline (attention)	10	0.7973	better, +0.0740
BKT	classical	50	0.7819	better, +0.0894
PFA	classical	50	0.7657	better, +0.1055

- **"Better"** is used only when a Holm-corrected paired test gives $p < 0.05$ *and* the bootstrap confidence interval of the difference excludes 0.
- **Repeated over 50 seeds** (`reports/v3/seeds.md`): the gated model beat the no-gate version on 50 of 50 matched seeds. The gain (+0.0073) is about 5 times the seed-to-seed spread (0.0015), so it is not luck.
- **Hints and attempts** are real inputs of the model and change its predictions in a sensible way. After a plain wrong answer, the predicted chance on the next question is 0.41. After a wrong answer that needed a second try, it is 0.58. Test AUC holds at the same level (0.8687; `reports/v3/meta_assist.md`).
- **Setup:** our models were trained for 15 epochs on all inputs, including response time, attempts and hints. The deep baselines were trained for 10 epochs on skill, correctness and difficulty.

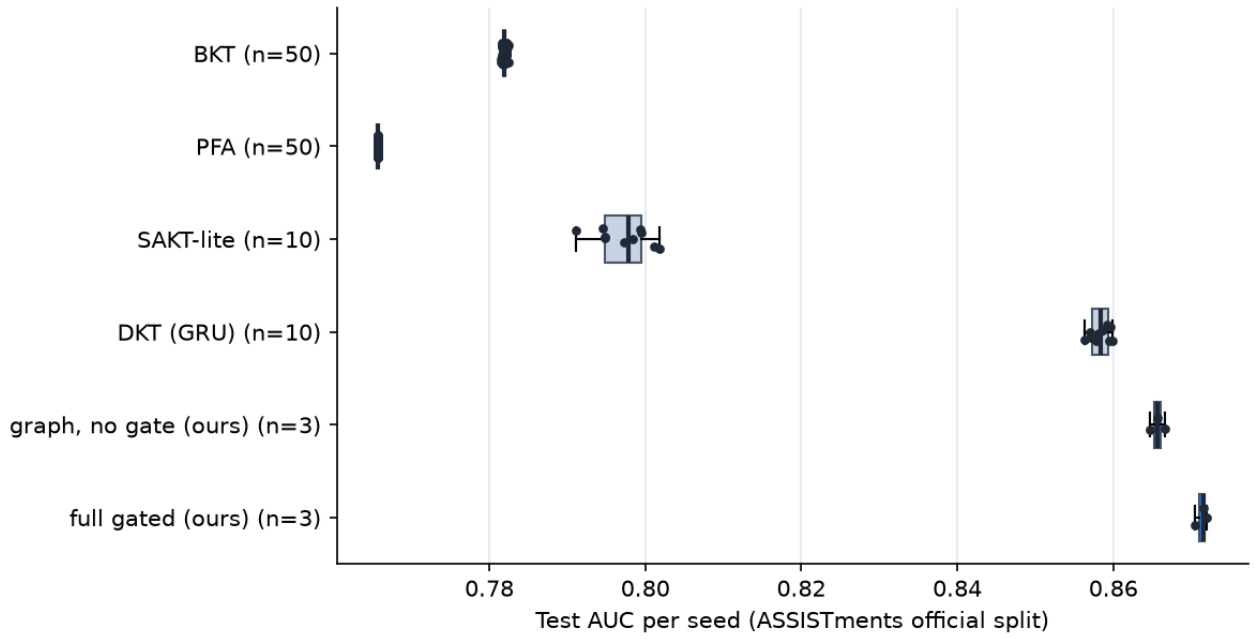


Figure 5. Test AUC for every trained run of every model. Each dot is one seed. The full gated model (bottom row) sits furthest to the right, and its runs are tightly grouped. Generated by `scripts/benchmark_baselines_v3.py`.

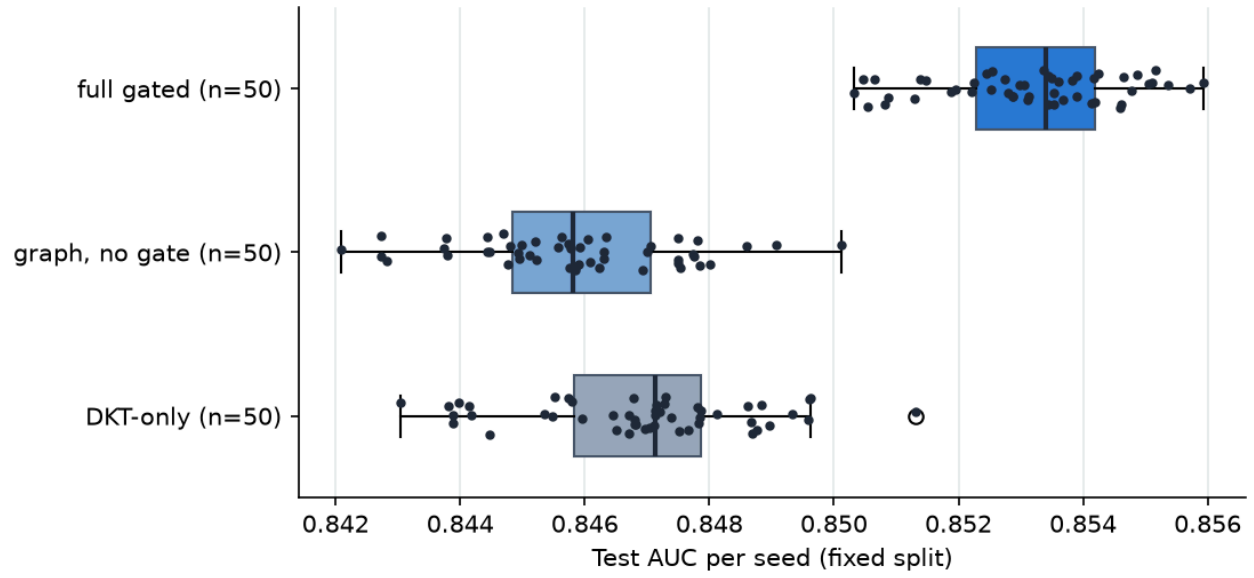


Figure 6. Fifty retraining runs per variant under the same 5-epoch budget. Every full gated run is above the best run of the version without the gate. Generated by `scripts/run_seeds.py`.

8 How to check that the work is genuine

Each point below can be checked in a few minutes on the project laptop. Paths are relative to the repository folder.

Public, citable data

The training data is the ASSISTments 2009 skill-builder dataset, a public dataset widely used in knowledge-tracing research. The raw file is kept unchanged in `data/raw/` (525,534 rows).

The model runs and checks itself

Running the model file loads the real data, runs a batch, and checks that `forward()` and the serving function give the same prediction (difference below 0.001).

```
python src/model.py
```

Automated tests

289 pytest tests pass. The only failing tests (15) cover the original fixed-quiz flow, which the adaptive system replaced. None were introduced by this work.

```
python scripts/run_tests_v3.py
```

Every number traces to a file

An audit script checks that every number in the documents appears in a generated report. The last run checked 321 numbers: 0 untraced, PASS.

```
python scripts/honesty_audit.py
```

Full history of the work

Git records every change with its date: 38 commits from 9 to 30 October 2026. Experiments were registered before they were run (for example `reports/v3/META_PREREG.md`).

```
git log --oneline
```

Live demo with the model's internals

Start the app and answer a few questions. The quiz page shows the predicted P before each question and the skill graph. The Model internals page plots the gate values for that session.

```
python scripts/start_demo.py
http://127.0.0.1:3000/
http://127.0.0.1:3000/model-
internals.html
```

Rigorous statistics

Comparisons use paired tests over matched runs, Holm correction for multiple comparisons, and 1,000-sample bootstrap intervals over students. A model is called "better" only when all of these agree.

Reproducible runs

Every experiment prints its seeds, saves one row per run as it goes, and records the exact command at the top of its report.

```
reports/v3/seeds.csv
reports/v3/benchmark_runs.csv
```

9 Contributions of this project

1. **An end-to-end adaptive loop.** The model predicts, the session engine chooses, the student answers, and the history updates. Every question carries a visible reason built from the model's numbers.

2. **A difficulty-aware mastery profile.** The model is asked "what would P be at Easy, Medium and Hard?" for each topic. A status is certified only where the student was actually tested near that level.

3. **Measured vs predicted knowledge.** Topics never asked are shown as "predicted, not measured" from the skill graph and checked with probe questions.

4. **Hints and second attempts as model inputs.** Help-seeking is recorded with each answer and fed into the model.

5. **A rigorous evaluation.** Four standard baselines, 50-seed variance study, pre-registered experiments, paired statistics and an automated audit of every number.

The model builds on established foundations: recurrent knowledge tracing (DKT), graph attention over related skills, and gated fusion of encoders. This project combines them into one model, trains and evaluates it carefully, and serves it in a working application.

10 Glossary

Knowledge tracing	Estimating what a student knows from their past answers, so as to predict the next one.
AUC	The chance that the model gives a correct answer a higher score than a wrong one. 0.5 is guessing, 1.0 is perfect.
GRU	Gated recurrent unit: a neural network layer that reads a sequence one step at a time and keeps a running memory.
Gate	A learned set of 64 weights between 0 and 1 that decides how much to trust recent answers versus the whole history.
GAT	Graph attention layer: lets each skill borrow information from related skills, weighting each neighbour by learned attention.
PPMI	Positive pointwise mutual information: a score for how much more often two skills appear together than chance. It sets the edge weights of the skill graph.
Seed	The number that fixes the random starting point of training. Repeating with many seeds shows how stable a result is.

Ablation	Removing one part of the model (for example the gate) and retraining, to measure what that part contributes.
Baselines	PFA and BKT (classical statistical models), DKT and SAKT (deep learning models) are the standard methods used for comparison.

Sources: docs/PROJECT_OVERVIEW.md, docs/ARCHITECTURE_NOTES_v3.md, reports/v3/benchmark.md, reports/v3/seeds.md, reports/v3/meta_assist.md and the source files named above.